



Univerzitet u Sarajevu
Elektrotehnički fakultet u Sarajevu
Odsjek za računarstvo i informatiku



Projektni zadatak – II dio

Ime i prezime: Zehra Buza

Broj indexa: 19273

Postavka zadatka:

Potrebno je odabrati jednu metodu iz konzolne aplikacije napisane za 1. dio projekta koja ima što veći indeks održavanja (npr. sadrži if iskaze, petlje, implementirane algoritme). Za tu metodu je potrebno izvršiti sljedeće:

1. Izračunati McCabe metriku i nacrtati kontrolni graf. Uporediti rezultate sa rezultatima iz Code Metrics alata i prodiskutovati kvalitet koda i dobijene rezultate.
2. Uraditi white box testiranje nad metodom koristeći primjenjiva 3 različita primjenjiva pristupa white box¹. Prikazati odabrane metode, dizajnirane testne slučajeve i provedeni postupak testiranja.
3. Izvršiti code tuning² za odabranu metodu primjenom minimalno 3 različite tehnike. Navesti koje su tehnike korištene i prikazati novi kod. Nakon svake urađene tehnike zabilježiti vrijeme izvršavanja i alociranu memoriju sa velikim brojem pokretanja. Zabilježiti i prodiskutovati i McCabe metriku i indeks održavanja te metode prije i poslije code tuning-a.
4. Nakon code tuning-a, izvršiti refaktoring³ tako da se postignu što bolje vrijednosti metrika. Prikazati kojoj stavki unutar checklist-e za refaktoring svaka promjena pripada, i kako ona utječe na metrike u sklopu Code Metrics.

1. Zadatak

Metoda: RegisterUser(User user)

```
// Registracija korisnika
17 references
public void RegisterUser(User user)
{
    foreach (var existingUser in users)
    {
        if (existingUser.Username == user.Username)
        {
            throw new ArgumentException("Korisničko ime već postoji.");
        }

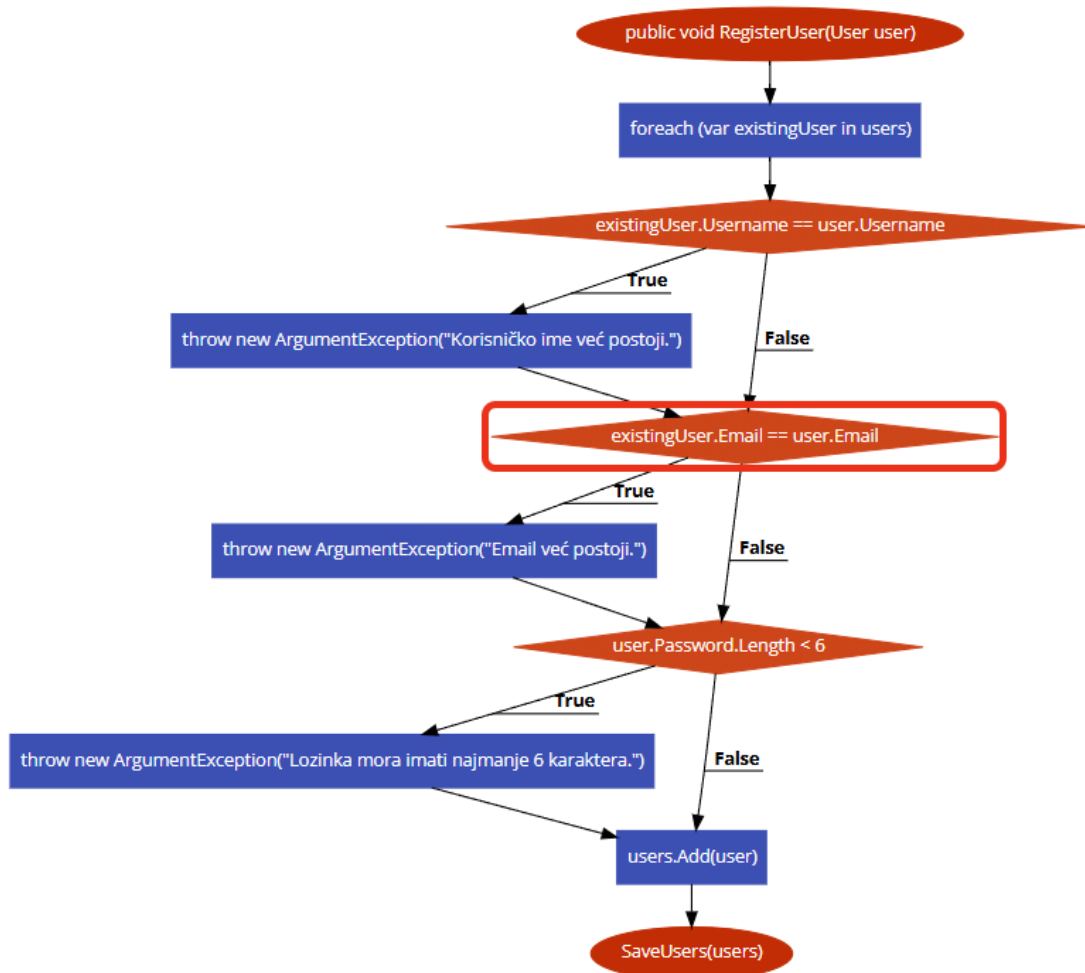
        if (existingUser.Email == user.Email)
        {
            throw new ArgumentException("Email već postoji.");
        }
    }

    if (user.Password.Length < 6)
    {
        throw new ArgumentException("Lozinka mora imati najmanje 6 karaktera.");
    }

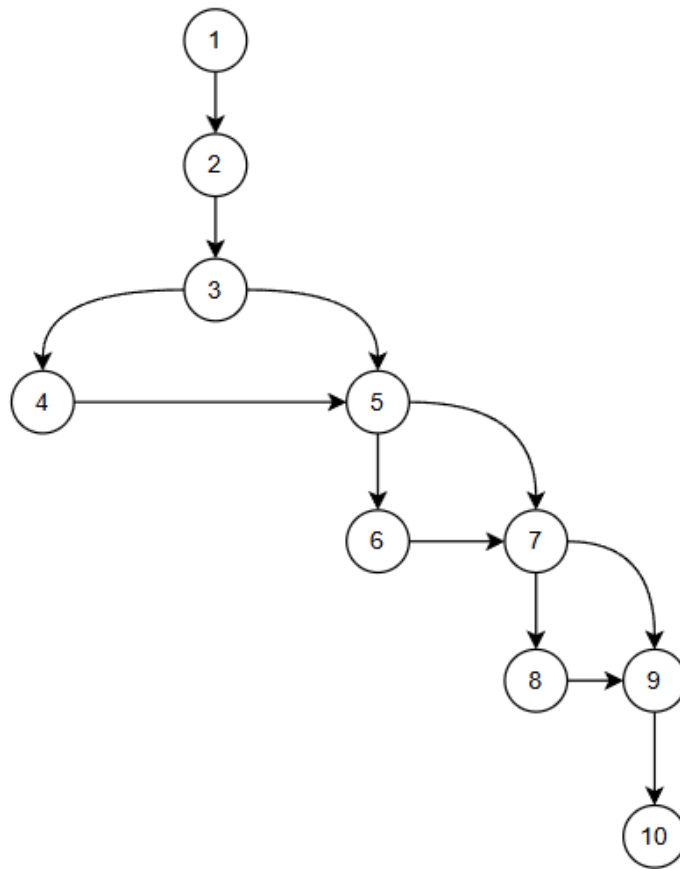
    users.Add(user);

    SaveUsers(users);
}
```

1. Dijagram toka



2. Kontrolni graf



3. Računanje McCabe metrike:

$e=12$

$n=10$

$p=1$

$$M = e - n + 2 * p = 12 - 10 + 2 * 1 = 2 + 2 = 4$$

Rezultati iz Code Metrics alata:

	Maintainability Index	Cyclomatic Complexity	Depth of Inheritance	Class Coupling	Lines of Source code	Lines of Executable code
RegisterUser(User) : void	63	5		4	29	9

Računanjem McCabe metrike dobijemo rezultat 4, dok u Code Metrics rezultat pokazuje 5. Iako bi ove vrijednosti trebale da budu iste, razlika nastaje zbog načina na koji Visual Studio analizira izraze u kodu. Vrijednost ciklomatske kompleksnosti je manja od 10, što ovaj kod čini poprilično jednostavnim za testiranje i održavanje. Pored toga, u rezultatima uočavamo

da je indeks održavanja 63, što se smatra održivim, ali ukazuje na mogućnost refaktorisanja kako bi se dodatno poboljšala održivost.
Class Coupling je nizak što također ukazuje na dobru održivost koda.

2. Zadatak

Redni broj puta	Čvorovi koje put obuhvata
1	1-2-3-4-5-6-7-8-9-10
2	1-2-3-4-5-6-7-9-10
3	1-2-3-4-5-7-8-9-10
4	1-2-3-4-5-7-9-10
5	1-2-3-5-6-7-8-9-10
6	1-2-3-5-6-7-9-10
7	1-2-3-5-7-8-9-10
8	1-2-3-5-7-9-10

Za obuhvat svih grana potrebna su nam 2 puta:

Put 1: 1-2-3-4-5-6-7-8-9-10

Put 8: 1-2-3-5-7-9-10

Koristit ću [TestInitialize] metodu, pomoću koje ću pripremiti podatke za obradu u nekim testovima:

```
[TestClass]
0 references
public class UserServiceTest
{
    private UserService _userService;

    [TestInitialize]
    0 references
    public void Setup()
    {
        // Kreiranje nove instance UserService prije svakog testa
        _userService = new UserService();
        var existingUser = new User("TestUser", "existing@example.com", "password123", new List<Task>());
        _userService.RegisterUser(existingUser);
    }
}
```

Obuhvat grana/odluka:

Da bi pokrivenost svih grana bila 100% potrebna su 4 testa. U prvom testu se testira da li sistem ispravno baca izuzetak (ArgumentException) kada se pokuša registrovati korisnik sa korisničkim imenom koje već postoji u bazi podataka korisnika. U drugom testu se provjerava da li sistem ispravno baca izuzetak (ArgumentException) kada se pokuša registrovati korisnik s email adresom koja već postoji. Treći test provjerava da li sistem ispravno baca izuzetak (ArgumentException) kada se korisnik pokuša registrovati sa lozinkom koja je prekratka. Četvrti test provjerava da li sistem ispravno registruje korisnika i dodaje ga u listu korisnika.

Test 1:

```
[TestMethod]
[ExpectedException(typeof(ArgumentException), "Korisničko ime već postoji.")]
0 references
public void TestUsernameAlreadyExists()
{
    var user = new User("TestUser", "test@example.com", "password123", new List<Task>());
    _userService.RegisterUser(user);
}
```

Test 2:

```
[TestMethod]
[ExpectedException(typeof(ArgumentException), "Email već postoji.")]
0 references
public void TestEmailAlreadyExists()
{
    var newUser = new User("uniqueUser", "existing@example.com", "password123", new List<Task>());
    _userService.RegisterUser(newUser);
}
```

Test 3:

```
[TestMethod]
[ExpectedException(typeof(ArgumentException), "Lozinka mora imati najmanje 6 karaktera.")]
0 references
public void TestPasswordTooShort()
{
    var newUser = new User("uniqueUser", "unique@example.com", "short", new List<Task>());
    _userService.RegisterUser(newUser);
}
```

Test 4:

```
[TestMethod]
0 references
public void TestSuccessfulRegistration()
{
    var newUser = new User("uniqueUser", "unique@example.com", "password123", new List<Task>());
    _userService.RegisterUser(newUser);

    // Provjera da li je korisnik uspješno registrovan
    var allUsers = _userService.GetUsers();
    Assert.AreEqual(2, allUsers.Count);
    Assert.AreEqual(newUser.Username, allUsers[1].Username);
}
```

Obuhvat linija :

Za obuhvat svih linija potrebna su nam 2 puta:

Put 1: 1-2-3-4-5-6-7-8-9-10

Put 8: 1-2-3-5-7-9-10

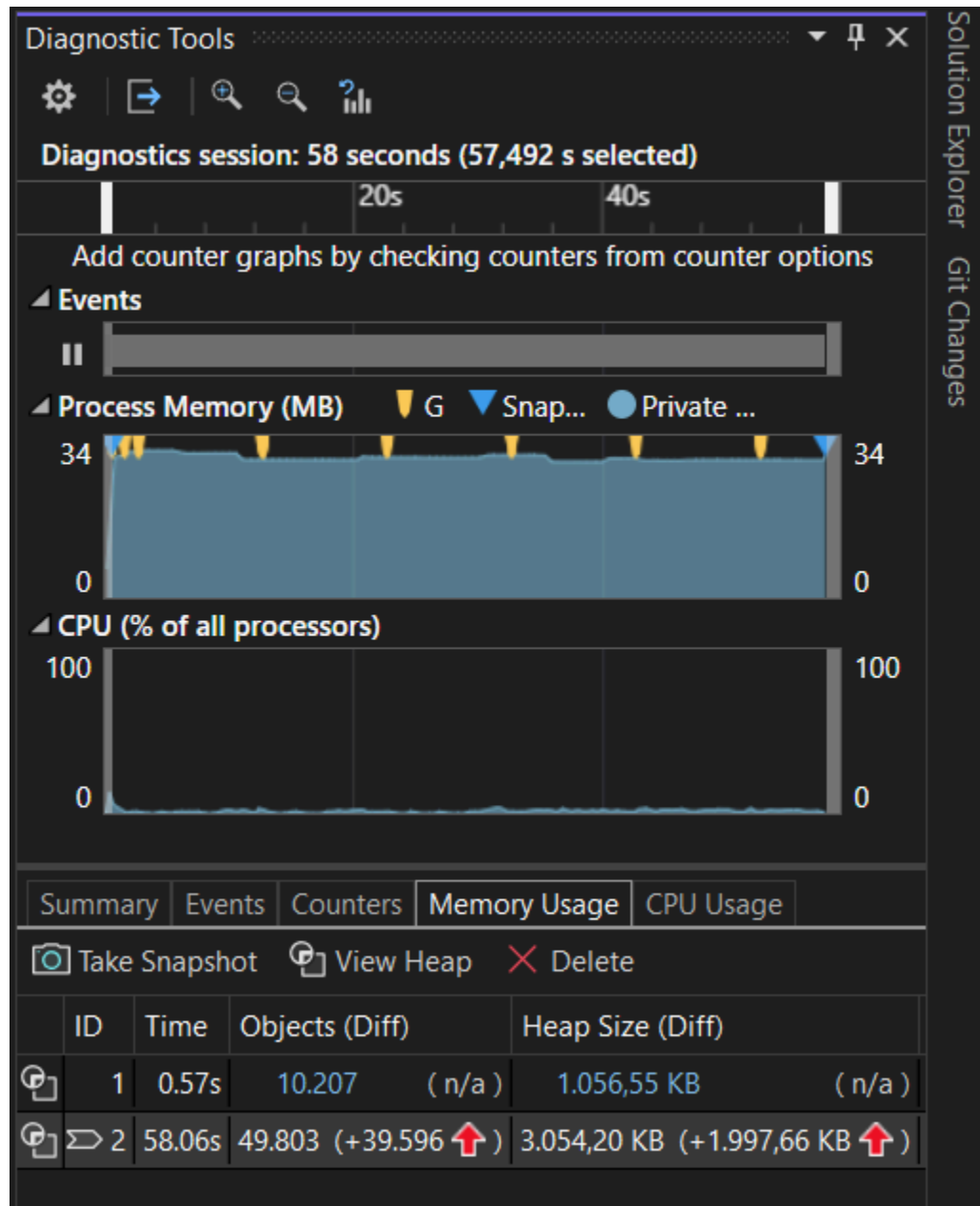
Referencirat ćemo se na testove iz prethodnog pristupa, s obzirom da se uz pomoć prethodnih testova napisanih za obuhvat grana mogu obuhvatiti i sve linije.

Obuhvat puteva :

Referencirat ćemo se na testove iz prvog pristupa, s obzirom da se uz pomoć testova napisanih za obuhvat grana mogu obuhvatiti i svi putevi.

3. Zadatak

Vrijeme izvršavanja i alocirane memorije prije promjene koda:



Vrijeme: 58.06-0.57=57.49

Alocirana memorija: 3054.20KB-1056.55KB = 1997.66

Podatke za McCabe metriku i indeks održavanja ove funkcije imamo u prvom zadatku.

Prva tehnika : Lazy Loading (Lazy Evaluation):

Lazy loading je tehnika koja se koristi da bi se resursi ili podaci učitali tek onda kada su stvarno potrebni, umesto odmah pri inicijalizaciji objekta.

Ovom tehnikom smo postigli učitavanje korisnika iz fajla samo kada se to zaista zahtijeva. Ako korisnici nisu potrebni podaci se neće učitavati, čime se štedi vrijeme i resursi, s obzirom da koristimo jako veliki broj podataka za testiranje.

```
// RegisterUser - Registracija korisnika
private bool _isUsersLoaded = false; // Flag za praćenje učitavanja korisnika
private List<User> _users;
6 references | 1/1 passing
public void RegisterUser(User user)
{
    // Lazy Loading: Učitavanje korisnika samo kada je potrebno
    if (!_isUsersLoaded)
    {
        _users = LoadUsers();
        _isUsersLoaded = true;
    }

    // Provjera za duplikate korisnika
    foreach (var existingUser in _users)
    {
        if (existingUser.Username == user.Username)
        {
            throw new ArgumentException("Korisničko ime već postoji.");
        }

        if (existingUser.Email == user.Email)
        {
            throw new ArgumentException("Email već postoji.");
        }
    }

    // Provjera minimalne dužine lozinke
    if (user.Password.Length < 6)
    {
        throw new ArgumentException("Lozinka mora imati najmanje 6 karaktera.");
    }

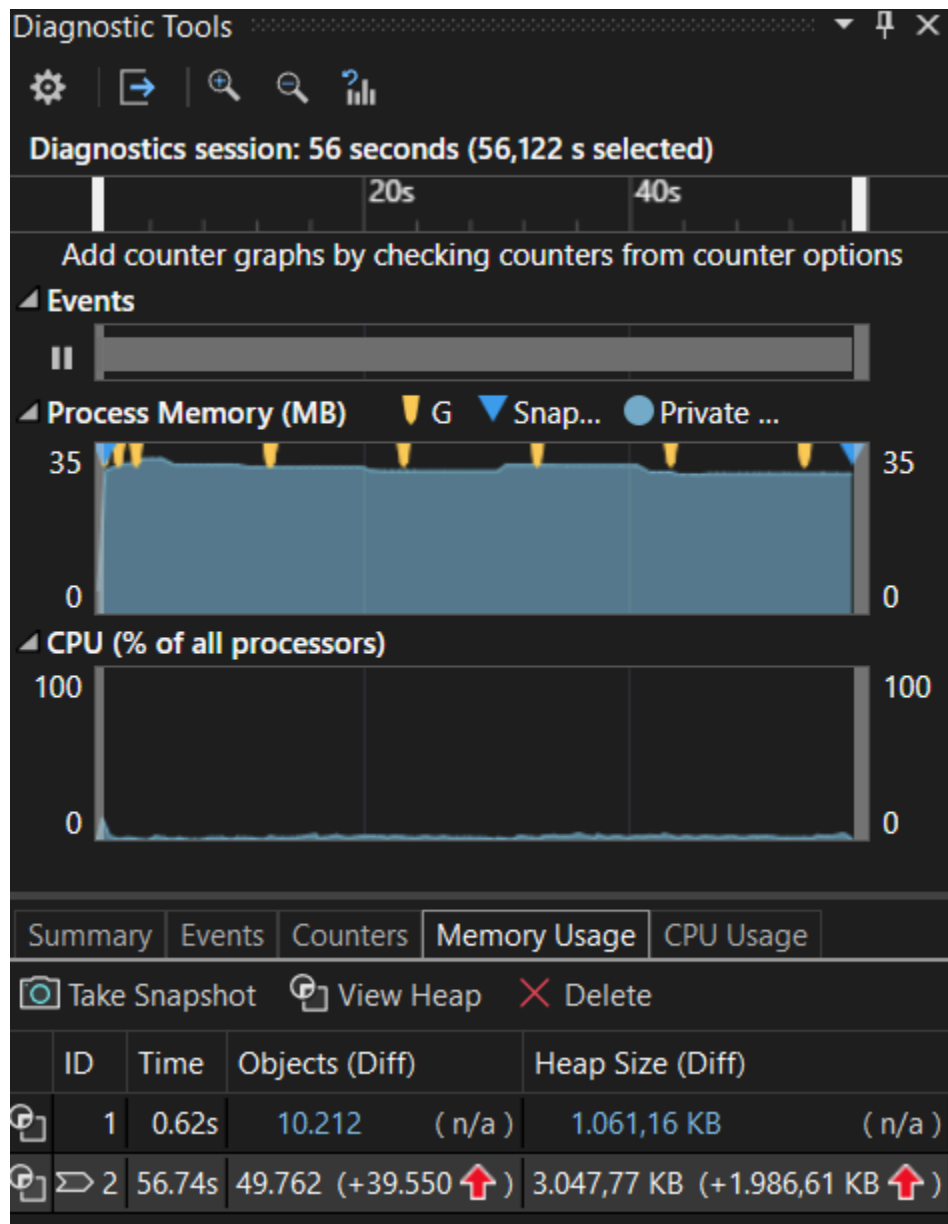
    // Dodavanje korisnika i čuvanje
    _users.Add(user);
    SaveUsers(_users);
}
```

McCabe metrika i indeks održavanja:

Hierarchy ^	Maintainability Index	Cyclomatic Complexity	Depth of Inheritance	Class Coupling
RegisterUser(User) : void	59	7		4

Nakon izmjene koda indeks održavanja je manji za 4 u odnosu na početni kod, dok je Cyclomatic Complexity veći za 2.

Vrijeme izvršavanja i alocirana memorija nakon code tuninga :



Vrijeme: 56.74s-0,62s = 56.12s

Alocirana memorija: 3047.77KB - 1061.16KB = 1986.61 KB

Nakon primjene lazy loading-a, vrijeme izvršavanja je kraće za 1.37 sekundi, što ukazuje na smanjenje ukupnog vremena potrebnog za obradu korisnika. Osim toga, potrebna alocirana memorija smanjila se sa 1997.66 KB na 1986.61 KB. Ovo smanjenje u potrošnji memorije ukazuje na efikasnije upravljanje resursima, što može doprinjeti većoj skalabilnosti aplikacije.

Druga tehnika : Minimiziranje rada unutar petlje

Minimiziranjem rada unutar petlje smanjujemo broj operacija u petlji. Prethodno su se unutar svake iteracije petlje iznova provodile dvije provjere za svakog korisnika (za korisničko ime i email). Takođe, za svaki poklapajući uslov generisala se nova poruka o grešci. Ove operacije su ponavljane i nepotrebno trošile procesorsko vrijeme.

Ovim postižemo da ako je lista korisnika velika, metoda postaje značajno brža jer se iteracije prekidaju čim se nađe prvo poklapanje.

```
public void RegisterUser(User user)
{
    // Lazy Loading: Učitavanje korisnika samo kada je potrebno
    if (users == null || !_isUsersLoaded)
    {
        users = LoadUsers();
        _isUsersLoaded = true;
    }

    // Provjera minimalne dužine lozinke prije ulaska u petlju
    if (user.Password.Length < 6)
    {
        throw new ArgumentException("Lozinka mora imati najmanje 6 karaktera.");
    }

    // Provjera za duplikate korisnika unutar jedne petlje, kombinovanjem uvjeta
    foreach (var existingUser in users)
    {
        if (existingUser.Username == user.Username || existingUser.Email == user.Email)
        {
            string error = existingUser.Username == user.Username
                ? "Korisničko ime već postoji."
                : "Email već postoji.";
            throw new ArgumentException(error);
        }
    }

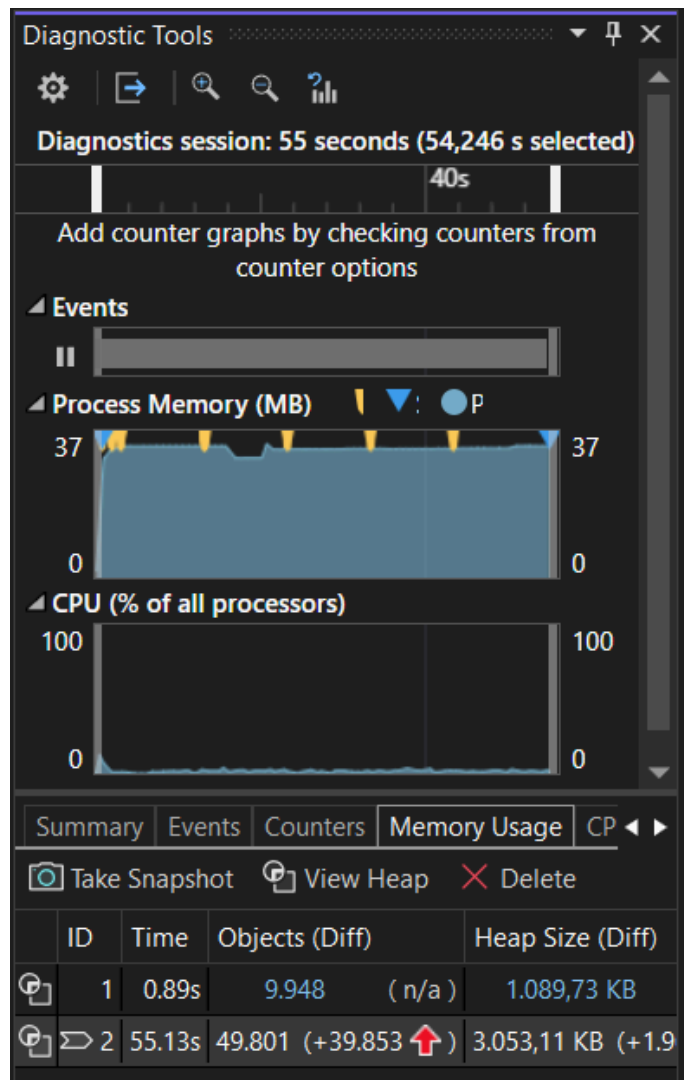
    // Dodavanje korisnika i čuvanje
    users.Add(user);
    SaveUsers(users);
}
```

McCabe metrika i indeks održavanja:

Hierarchy ^	Maintainability Index	Cyclomatic Complexity	Depth of Inheritance	Class Coupling
RegisterUser(User) : void	59	8		4

Nakon izmjene koda indeks održavanja je manji za 4 u odnosu na početni kod, dok je Cyclomatic Complexity veći za 3.

Vrijeme izvršavanja i alocirana memorija nakon code tuninga :



Vrijeme: 55.13s-0,89s = 54.24s

Alocirana memorija: 3053.11KB - 1089.73KB = 1963.38 KB

Nakon primjene minimiziranja rada unutar petlje, vrijeme izvršavanja je kraće za 3.25 sekundi. Potrebna alocirana memorija smanjila se sa 1997.66 KB na 1963.38 KB. Smanjenje memorijske potrošnje ukazuje na efikasniju upotrebu resursa, što je posebno značajno kod velikih setova podataka.

Treća tehnika : Korištenje hešinga

```
private HashSet<string> usernames = new HashSet<string>();
private HashSet<string> emails = new HashSet<string>();

5 references | 1/1 passing
public void RegisterUser(User user)
{
    // Lazy Loading: Učitavanje korisnika samo kada je potrebno
    if (users == null || !_isUsersLoaded)
    {
        users = LoadUsers();
        _isUsersLoaded = true;

        // Popunjavanje heš tabela za brzu provjeru
        foreach (var existingUser in users)
        {
            usernames.Add(existingUser.Username);
            emails.Add(existingUser.Email);
        }
    }

    // Provjera minimalne dužine lozinke prije ulaska u petlju
    if (user.Password.Length < 6)
    {
        throw new ArgumentException("Lozinka mora imati najmanje 6 karaktera.");
    }

    // Provjera za duplikate unutar petlje
    foreach (var existingUser in users)
    {
        if (existingUser.Username == user.Username || existingUser.Email == user.Email)
        {
            string error = existingUser.Username == user.Username
                ? "Korisničko ime već postoji."
                : "Email već postoji.";
            throw new ArgumentException(error);
        }
    }

    // Dodavanje korisnika u listu i ažuriranje heš tabela
    users.Add(user);
    usernames.Add(user.Username);
    emails.Add(user.Email);

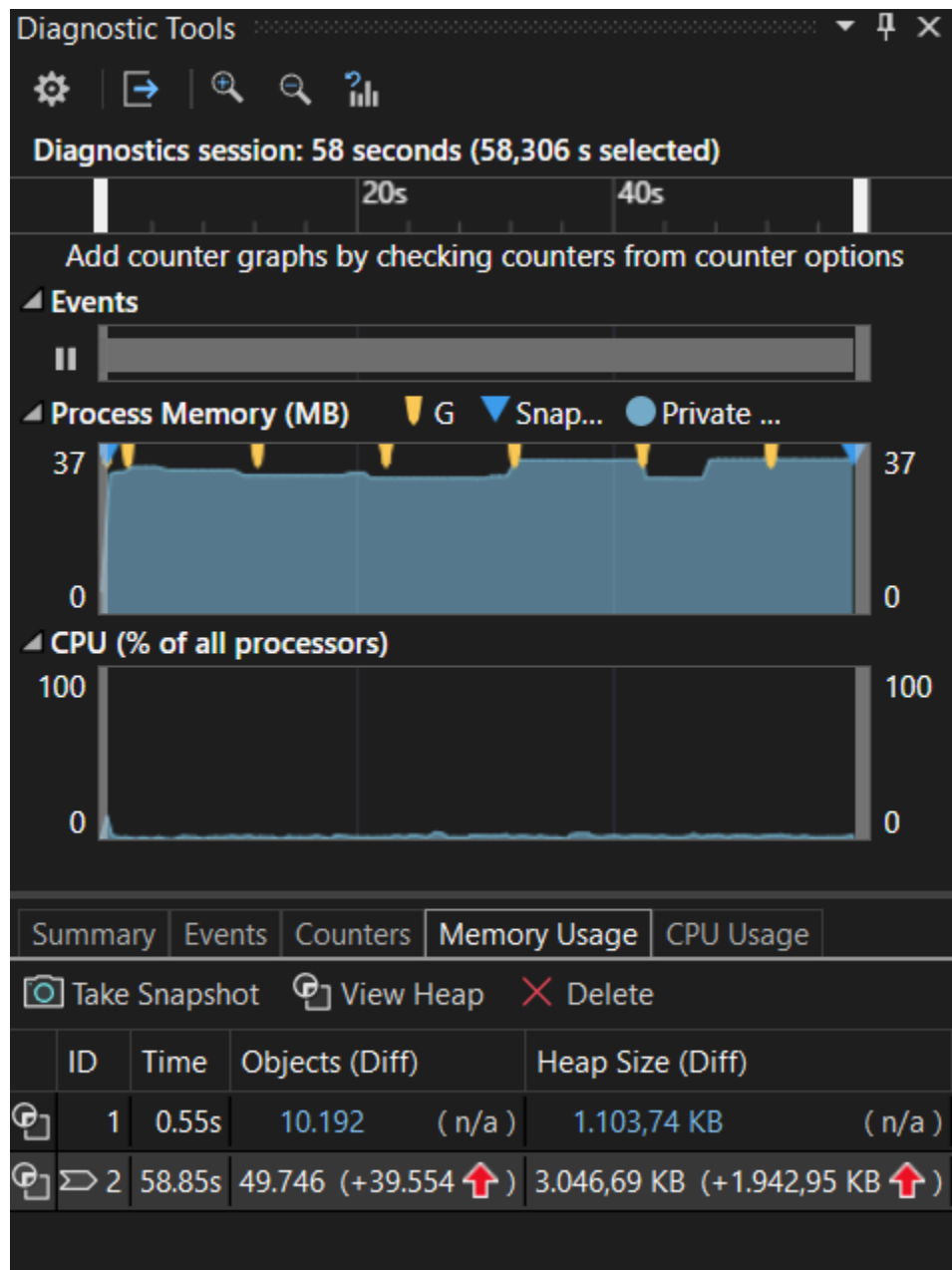
    SaveUsers(users);
}
```

McCabe metrika i indeks održavanja:

Hierarchy	Maintainability Index	Cyclomatic Complexity	Depth of Inheritance	Class Coupling
RegisterUser(User) : void	54	9		5

Nakon izmjene koda indeks održavanja je manji za 9 u odnosu na početni kod, dok je Cyclomatic Complexity veći za 4.

Vrijeme izvršavanja i alocirana memorija nakon code tuninga



Vrijeme: 58.85s-0,55s = 58.3s

Alocirana memorija: 3046.69KB - 1103.74KB = 1942.95KB

Nakon primjene korištenja hešinga, vrijeme izvršavanja je duže za 0.81 sekundi. Potrebna alocirana memorija smanjila se sa 1997.66 KB na 1942.95 KB. Iako je došlo do blagog povećanja vremena, smanjenje memorijske potrošnje ukazuje na efikasniju upotrebu resursa, što je posebno značajno kod velikih setova podataka.

4. Zadatak

Kod nakon izvršenog refaktoringa:

```
// Registracija korisnika
private const int MinimumPasswordLength = 6;
private HashSet<string> usernames = new HashSet<string>();
private HashSet<string> emails = new HashSet<string>();

5 references | 1/1 passing
public void RegisterUser(User user)
{
    // Validacija lozinke
    ValidatePassword(user.Password);

    // Provjera duplikata korisničkog imena i emaila
    ValidateDuplicate(user);

    // Dodavanje korisnika
    users.Add(user);
    usernames.Add(user.Username);
    emails.Add(user.Email);

    SaveUsers(users);
}

1 reference
private void ValidatePassword(string password)
{
    if (password.Length < MinimumPasswordLength)
    {
        throw new ArgumentException("Lozinka mora imati najmanje 6 karaktera.");
    }
}

1 reference
private void ValidateDuplicate(User user)
{
    if (usernames.Contains(user.Username))
    {
        throw new ArgumentException("Korisničko ime već postoji.");
    }

    if (emails.Contains(user.Email))
    {
        throw new ArgumentException("Email već postoji.");
    }
}
```

Code Metrics:

Hierarchy ▲	Maintainability Index	Cyclomatic Complexity
RegisterUser(User) : void	70	1

Nakon primjene refaktoringa, postigli smo indeks održivosti od 70, što označava vrlo čitljiv i lako održiv kod. Ipak, primjenom refaktoringa ciklomatska kompleksnost smanjena je na 1, što je manje od inicijalne vrijednosti i bilo kojeg drugog koda koji smo imali kroz code tuning. Smanjenje ciklomatske kompleksnosti rezultat je izdvajanja logičkih uslova i njihovog pojednostavljivanja kroz metode ValidatePassword i ValidateDuplicate.

Check liste:

1. Refaktoring na nivou podataka

CHECKLIST: Data Level Refactorings

- ☒ Replace a magic number with a named constant.
- ☐ Rename a variable with a clearer or more informative name.
- ☐ Move an expression inline.
- ☐ Replace an expression with a routine.
- ☐ Introduce an intermediate variable.
- ☐ Convert a multi-use variable to a multiple single-use variables.
- ☐ Use a local variable for local purposes rather than a parameter.
- ☐ Convert a data primitive to a class.
- ☐ Convert a set of type codes to a class.
- ☐ Convert a set of type codes to a class with subclasses.
- ☒ Change an array to an object.
- ☒ Encapsulate a collection.
- ☐ Replace a traditional record with a data class.
- ☐ ...

2. Refaktoring na nivou iskaza

CHECKLIST: Statement Level Refactorings

- ☒ Decompose a boolean expression.
- ☐ Move a complex boolean expression into a well-named boolean function.
- ☐ Consolidate fragments that are duplicated within different parts of a conditional.
- ☐ Use break or return instead of a loop control variable.
- ☐ Return as soon as you know the answer instead of assigning a return value within nested if-then-else statements.
- ☐ Replace conditionals with polymorphism (especially repeated case statements).
- ☐ Create and use null objects instead of testing for null values.
- ☐

3. Refaktoring na nivou metoda

CHECKLIST: Routine Level Refactorings

- ☒ Extract a routine.
- ☐ Move a routine's code inline.
- ☐ Convert a long routine to a class.
- ☐ Substitute a simple algorithm for a complex algorithm.
- ☐ Add a parameter.
- ☐ Remove a parameter.
- ☒ Separate query operations from modification operations.
- ☐ Combine similar routines by parameterizing them.
- ☐ Separate routines whose behavior depends on parameters passed in.
- ☐ Pass a whole object rather than specific fields.
- ☐ Pass specific fields rather than a whole object.
- ☐ Encapsulate downcasting.
- ☐